ARM Connect - Merchant Platform

Integration Guide

Registration · Login · Onboarding · Products

One end-to-end view of the merchant journey, from creating an account to buying investment products on behalf of members.

Merchant API · Onboarding API · Product API



Quick start

The shortest path from nothing to an authenticated product call. Everything else in this guide is detailed on these steps.

What you need before you write any code

Item	Where it comes from	Notes
{{baseurl}}	ARM — sandbox or production host	Product endpoints hang off this. Onboarding uses https://sandapi.arm.com.ng/api/MerchantOnboarding
Merchant credentials	You create them — Part 1	Email + password. Registration also returns an ApiKey and SecretKey
Bearer token	POST /api/v1/merchant/login	Every call below carries it. Refresh before expiresAt
Merchant type	The merchantType claim in your token	B2B or B2B2C. It decides whether Virtual Accounts are available to you

The happy path, in curl

```
# 1. Check the name and email are free — an OTP is emailed to you
curl -X POST "$BASE/api/v1/merchant/validate" -H "Content-Type: application/json" -d '{
  "businessName": "Acme Corp", "email": "merchant@acme.com",
  "firstName": "John", "lastName": "Doe", "phoneNumber": "+2348012345678" }'

# 2. Verify the OTP  (purpose MUST be Register)
curl -X POST "$BASE/api/v1/otp/validate" -H "Content-Type: application/json" \
  -d '{"email":"merchant@acme.com","otp":"123456","otpPurpose":"Register"}'

# 3. Register — capture secretKey from the response, it is shown ONCE
curl -X POST "$BASE/api/v1/merchant/register" -H "Content-Type: application/json" -d '{
  "request": {
    "businessName": "Acme Corp", "contactEmail": "merchant@acme.com",
    "firstName": "John", "lastName": "Doe", "contactPhone": "+2348012345678",
    "password": "SecureP@ssword123", "callbackUrl": "https://acme.com/webhook" } }'

# 4. Log in — everything from here needs this token
TOKEN=$(curl -s -X POST "$BASE/api/v1/merchant/login" \
  -H "Content-Type: application/json" \
  -d '{"email":"merchant@acme.com","password":"SecureP@ssword123"}' | jq -r .data.token)

# 5. Onboard the business — five sections, in any order, or all at once:
curl -X POST "$BASE/api/MerchantOnboarding/onboard-merchant" \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d @onboarding-payload.json

# 6. Poll until CRM approves you (or handle the webhook on your callbackUrl)
curl "$BASE/api/MerchantOnboarding/progress" -H "Authorization: Bearer $TOKEN"

# 7. Approved. Now you can transact:
curl "$BASE/Product/MutualFunds" -H "Authorization: Bearer $TOKEN"

curl -X POST "$BASE/Product/mutualfunds/purchase/bulk" \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"requests":[{"amount":2000,"productCode":"ARMMMF","membershipId":"3005575"}]}'
```

Five things that will trip you up

#	Gotcha	What to do
1	merchantEmail in the body must equal the email claim in your JWT	Never take it from user input — read it from the token you are already holding
2	The 200 from POST /documents means queued, not uploaded	The S3 write happens in the background. Confirm via GET /progress
3	A customer documents array replaces every existing document	To add one file, resend them all
4	A 200 on a bulk purchase or withdrawal does not mean every request in it succeeded	Inspect each element of data — each carries its own status and message
5	Virtual Accounts are B2B only	If your merchantType is B2B2C, do not build against /Product/VirtualAccounts — it will fail authorisation

The big picture

Six stages. Each one unlocks the next — you cannot onboard without a token, and you cannot transact without CRM approval.

What changes at each gate

After you…	You hold	Which unlocks
Register	MerchantId, ApiKey, SecretKey	Logging in
Log in	JWT (email, merchantType claims) + refresh token	Every onboarding and product call
Complete onboarding	Application auto-submitted to CRM, status Submitted	Waiting for approval
Get approved (webhook)	MembershipId, ZaniballId, status Approved	Customer onboarding and products
Onboard a customer	CrmAccountNumber, customer wallet	Transacting on their behalf (B2B2C)

THE ONE RULE TO REMEMBER

Identity always comes from the JWT, never from the request body. The merchantEmail you send must match the email claim in your token, or the server rejects the call. The body field is a confirmation, not an input.

Part 1 — Registration

Three calls, in order. The OTP cannot be skipped: the register endpoint checks that the email was verified before it will create anything.

Step 1 — Validate and trigger the OTP

POST `/api/v1/merchant/validate`

Checks that the business name and email are not already taken. If both are free, an OTP is emailed to the contact address.

```
{
  "businessName": "Acme Corp",
  "email": "merchant@acme.com",
  "firstName": "John",
  "lastName": "Doe",
  "phoneNumber": "+2348012345678"
}
```

Result	Meaning
200 OK	otpSent: true — proceed to step 2
409 Conflict	Business name or email already exists. The response tells you which field conflicts.

Step 2 — Validate the OTP

POST `/api/v1/otp/validate`

Submit the code with otpPurpose: "Register". On success the OTP is marked validated and the email is trusted for registration. An invalid or expired code returns 400 — request a new one via step 1.

Step 3 — Complete registration

POST `/api/v1/merchant/register`

```
{
  "request": {
    "businessName": "Acme Corp",
    "contactEmail": "merchant@acme.com",
    "firstName": "John",
    "lastName": "Doe",
    "contactPhone": "+2348012345678",
    "password": "SecureP@ssword123",
    "callbackUrl": "https://acme.com/webhook"
  }
}
```

Behind the scenes the service generates a unique MerchantId, an ApiKey and SecretKey pair, encrypts the password, persists the merchant, and creates an external service client with the merchant.access permission.

```
{
  "status": true,
```

```
  "statusCode": 200,
  "message": "Merchant registered successfully.",
  "data": {
    "merchantId": "abc123...",
    "apiKey": "...",
    "secretKey": "...",
    "createdAt": "2026-07-13T10:00:00Z"
  }
}
```

STORE THE SECRET KEY NOW

The secretKey is returned exactly once, at registration. Persist it securely before you close the response — there is no endpoint to retrieve it again.

CALLBACK URL

callbackUrl is where the platform pushes CRM approval and rejection events later — for both your own application and your customers'. Set it now; it is how you find out you have been approved without polling.

Errors you may hit on /register

Code	Cause	Fix
400	Missing businessName or password	Complete the payload
409	Email already registered	Log in instead, or use a different email
409	Business name already a service client	Pick a different business name
401	OTP was never validated for this email	Go back to steps 1 and 2

Part 2 — Login and authentication

One call, one token. Every endpoint after this point expects Authorization: Bearer <JWT>.

POST `/api/v1/merchant/login`

```
{
  "email": "merchant@acme.com",
  "password": "SecureP@ssword123"
}
```

```
{
  "status": true,
  "statusCode": 200,
  "message": "Login successful.",
  "data": {
    "token": "<JWT>",
    "refreshToken": "<refresh token>",
    "expiresAt": "2027-07-13T10:00:00Z",
    "merchantId": "abc123...",
    "businessName": "Acme Corp",
    "contactEmail": "merchant@acme.com",
    "apiKey": "...",
    "membershipId": "membership-456",
    "merchantType": "B2B",
    "zanibalId": null
  }
}
```

A wrong email and a wrong password both return the same 400, "Invalid email or password". That is deliberate: it prevents attackers from discovering which emails are registered.

Your merchant type is decided at login

The service checks whether you have any records in MerchantCustomerOnboardingData and stamps the answer into the token so downstream services can gate features.

Type	Condition	What it means for you
B2B	Merchants table only	You transact for your own business. Virtual Accounts are available to you. Customer onboarding does not apply.
B2B2C	Merchants and MerchantCustomerOnboardingData	You onboard and transact on behalf of individual customers. Virtual Accounts are NOT available to you.

THIS IS AUTOMATIC

You do not choose your type — you become B2B2C the moment you onboard your first customer. Log in again afterwards and your token will say so.

IT DETERMINES YOUR PRODUCT SURFACE

> Virtual Accounts are strictly a B2B feature. A B2B2C or B2C merchant calling /Product/VirtualAccounts should expect to fail authorisation checks. Confirm your merchant type before building against those endpoints.

What is inside the token

Claim	Value
client_name	Business name (falls back to MerchantId)
is_service	"true"
email	Contact email — the identity every onboarding endpoint reads
merchantId	Unique merchant identifier
membershipId	Membership ID (populated after CRM approval)
merchantType	"B2B" or "B2B2C"
zanibalId	ZanibalId, if present
permission	"merchant"
jti	Unique token ID (required by /create)

Part 3 — Merchant onboarding

Registering gave you keys. Onboarding gets you approved — a real CRM client account that can hold and invest money. Five sections of data, submitted in any order, auto-shipped to CRM the moment the last one lands.

> **SAVE AND CONTINUE**
>
> Sections are tracked as bits in CompletedSections. Save them in whatever order suits your UI, across as many sessions as you like. Nothing is lost, and nothing submits early.

Before you start

- Base URL: https://<host>/api/MerchantOnboarding
- Every call needs Authorization: Bearer <token>.
- The merchantEmail in each body must match the token's email claim.

The five sections

Bit	Section	Endpoint	Contains
0	Corporate Profile	POST /corporate	Legal name, RC, TIN, sector, registered address
1	Contact & Bank	POST /contact-bank	Contact person and corporate bank account
2	Signatories	POST /signatories	One call per signatory. Bit set on the first save
3	Mandate & Declarations	POST /mandate-declarations	Signing mandate, signature image, consents
4	Documents	POST /documents	CAC certificate, MEMART, utility bill

Step 0 — Create the CRM potential client (one time)

POST `/api/MerchantOnboarding/create`

A separate, one-off call that stands your merchant up at the CRM level as a "potential client". Independent of the step-by-step flow below. Requires a jti claim in the token, and emailAddress must equal the JWT email. Returns 201 with merchantId and membershipId.

```
{
  "accountName": "Acme Corp Ltd",
  "emailAddress": "contact@acme.com",
  "rcNumber": "RC123456",
  "tinVat": "12345678-0001",
  "mobilePhone": "08012345678",
  "source": "55"
}
```

Section 0 — Corporate details and address

POST `/api/MerchantOnboarding/corporate`

```
{
```

```
  "merchantEmail": "contact@acme.com",
  "legalBusinessName": "Acme Corp Ltd",
  "rcNumber": "RC123456",
  "tinVat": "12345678-0001",
  "natureOfBusiness": "Technology",
  "sector": "FinTech",
  "companyWebsite": "https://acme.com",
  "countryOfOperation": "Nigeria",
  "incorporationDate": "2010-05-01",
  "numberOfEmployees": "51-200",
  "averageAnnualTurnover": "100M-500M",
  "yearOfEstablishment": "2010",
  "corporateCategoryOfBusiness": "Private Limited",
  "addressNumber": "1",
  "registeredAddress": "15 Marina Street",
  "city": "Lagos",
  "state": "Lagos",
  "country": "Nigeria",
  "postalCode": "100001"
}
```

Writes the CorporateProfile and CorporateAddress sections together, and marks bit 0.

> **DO NOT FREE-TEXT THE DROPDOWNS**
>
> Pull valid values from the lookup endpoints so CRM accepts them: GET /average-turnover · /business-categories · /industry · /establishment-years · /number-of-employees

Section 1 — Contact person and bank account

POST `/api/MerchantOnboarding/contact-bank`

```
{
  "merchantEmail": "contact@acme.com",
  "contactName": "John Doe",
  "positionDesignation": "CEO",
  "emailAddress": "john@acme.com",
  "phoneNumber": "08012345678",
  "bankName": "First Bank",
  "accountName": "Acme Corp Ltd",
  "accountNumber": "1234567890",
  "bankVerificationNumber": "12345678901",
  "bankCode": "011",
  "nibbsCode": "000016"
}
```

Writes ContactDetails and BankAccountDetails, and marks bit 1. This BVN is also the fallback used later for customer wallet creation.

Section 2 — Signatories, one call each

POST `/api/MerchantOnboarding/signatories`

```
{
  "merchantEmail": "contact@acme.com",
  "orderIndex": 1,
  "fullLegalName": "Jane Doe",
  "designation": "Director",
```

```
  "gender": "Female",
  "dateOfBirth": "1980-03-15",
  "phoneNumber": "08099999999",
  "emailAddress": "jane@acme.com",
  "personalAddress": "5 Victoria Island",
  "isPrimary": true,
  "idType": "NationalId",
  "idNumber": "AB1234567",
  "idIssueDate": "2019-01-01",
  "idExpiryDate": "2029-01-01",
  "nin": "12345678901",
  "bvn": "22345678901",
  "utilityBillType": "NEPA"
}
```

- Adding more: call the endpoint again with the next signatory.
- Updating one: include signatoryId in the body.
- Listing them: GET /api/MerchantOnboarding/signatories

Bit 2 is marked as soon as the first signatory saves.

Section 3 — Mandate and declarations

POST **/api/MerchantOnboarding/mandate-declarations**

```
{
  "merchantEmail": "contact@acme.com",
  "mandateInstruction": "Any two signatories",
  "otherDescription": null,
  "signatureADataUrl": "...",
  "signatureDate": "2024-07-10",
  "confirmedInformationAccuracy": true,
  "acceptedTermsOfService": true,
  "acknowledgedFinancialRisks": true,
  "consentedToDataSharing": true,
  "isPoliticallyExposedPerson": false,
  "interestedInDirectDebitMandate": false,
  "acceptedMasterServiceAgreement": true,
  "agreementFullName": "Jane Doe",
  "agreementPhoneNumber": "08099999999"
}
```

signatureADataUrl accepts either a base64 data URL (decoded and uploaded to S3 for you) or an existing https:// S3 URL (stored as-is). Marks bit 3.

Section 4 — Documents

POST **/api/MerchantOnboarding/documents**

```
{
  "merchantEmail": "contact@acme.com",
  "documents": [
    { "documentType": "CACCertificate", "fileBase64": "JVBERi0x...",
      "fileName": "cac_certificate.pdf", "contentType": "application/pdf",
      "issueDate": "2010-05-01", "expiryDate": null },
    { "documentType": "MEMART", "fileBase64": "JVBERi0x...",
      "fileName": "memart.pdf", "contentType": "application/pdf" },
    { "documentType": "UtilityBill", "fileBase64": "JVBERi0x...",
      "fileName": "utility_bill.pdf", "contentType": "application/pdf",
```

```
      "utilityBillType": "NEPA", "issueDate": "2024-01-01",
      "expiryDate": "2025-01-01" }
  ]
}
```

THE 200 HERE MEANS QUEUED, NOT UPLOADED

This endpoint returns immediately; the S3 upload happens in the background. Do not treat the 200 as proof the file landed — poll GET /progress or GET /full-application to confirm. Base64 is never persisted to the database, only the resulting S3 URL. Marks bit 4.

Prefer one call? Use the draft endpoint

POST `/api/MerchantOnboarding/onboard-merchant`

```
{
  "merchantEmail": "contact@acme.com",
  "corporateDetails": { ... },
  "contactAndBank": { ... },
  "signatories": [ { ... }, { ... } ],
  "mandateAndDeclarations": { ... },
  "documents": [ { ... } ]
}
```

Every section is optional, and anything you omit is preserved from earlier saves. Same save-and-continue semantics, same auto-submit trigger — just fewer round trips.

Watching your progress

GET `/api/MerchantOnboarding/progress`

```
{
  "data": {
    "status": "InProgress",
    "currentSection": "Documents",
    "completedSections": 4,
    "totalSections": 6,
    "progressPercentage": 67,
    "submittedAt": null
  }
}
```

Ideal for driving a progress bar. For the full picture — every section, every document, in one payload — use GET /full-application.

Auto-submit to CRM

When the last section completes, without you doing anything, the service:

- Loads all section data from the database
- Pulls your documents back from S3 and re-encodes them as base64
- Builds the MerchantData payload and calls CreateMerchantClientAsync

- On success: status becomes Submitted, and MembershipId, MerchantId and ApprovalStatus are written back from the CRM response
- On failure: returns 422 with the CRM's reason

If it does not fire, or you need to retry after a 422, force it with POST /api/MerchantOnboarding/submit.

If a document gets rejected

Step	Who	Call	Effect
1	Admin	POST /documents/reject	Sets IsRejected, Status = Rejected, RejectionReason, RejectedAt on each document. Application status becomes UnderReview.
2	Merchant	GET /documents/rejected	Returns document type, file name, S3 URL, rejectionReason and rejectedAt — enough to show exactly what to fix.
3	Merchant	POST /documents	Re-upload the corrected files. Nothing else is needed.

Approval arrives by webhook

POST `/api/MerchantOnboarding/merchant-approval` · `/merchant-crm-webhook`

CRM pushes the outcome back to the platform, which updates your record and then forwards the payload to your CallbackURL. Routing is by membershipId: customer records are checked first, then merchants, and a 404 is returned if neither matches.

```
{
  "membershipId": "MEM123456",
  "approvalStatus": "Approved",
  "merchantId": "SCHEME-GUID",
  "zanibalId": "ZAN123456",
  "email": "contact@acme.com",
  "bvn": "22345678901"
}
```

/merchant-crm-webhook handles both outcomes: if approvalStatus is "Rejected" and a rejectionReason is present, the rejection path runs — status set to Rejected, failure reason stored, and an email sent listing the rejected documents and why.

YOU ARE NOW APPROVED

Once ApprovalStatus = Approved and MembershipId is populated, you can onboard customers (Part 4) and transact on products (Part 5).

Part 4 — Client onboarding (B2B2C)

Bring your own customers into the ecosystem. Six sections, one endpoint, called as many times as you like — the record ships to CRM by itself the moment the sixth section lands.

> **PREREQUISITES — BOTH ARE HARD GATES**
>
> Your merchant record must be Approved, and your MembershipId must be non-null (populated by the CRM approval webhook). Without both, customer onboarding will not work.

POST `/api/MerchantOnboarding/merchant-customers/onboard`

You are identified by your JWT's email claim; the customer is identified by customerEmail in the body. Only the sections present in a given request are written — everything else is left untouched.

The six sections

Bit	Section	Key fields
0	Personal Details	Name, DOB, gender, nationality, occupation, income range, risk appetite and profile, source of funds, mother's maiden name
1	KYC Details	BVN and validation status, ID type / number / dates, NIN, utility bill type and dates
2	Address	Address number, street, town, country, state code
3	Next of Kin	Name, gender, DOB, relationship type, phone, email, address
4	Bank Details	Bank name, account name and number, bank code, branch code, NIBSS code
5	Documents	Base64 files in folders: passport, bvn_passport, id_card, utility_bill

Example — two sections in one call

```
{
  "customerEmail": "customer@example.com",
  "personalDetails": {
    "surname": "Smith", "firstName": "Alice", "middleName": "Grace",
    "dateOfBirth": "1990-06-15", "gender": "Female",
    "nationality": "Nigerian", "title": "Ms",
    "stateOfOrigin": "Lagos", "placeOfBirth": "Lagos",
    "annualExpectedAnnualIncomeRange": "500K-1M",
    "riskAppetite": "Medium", "riskProfile": "Balanced",
    "sourceOfFunds": "Salary", "occupation": "Engineer",
    "mothersMaidenName": "Johnson",
    "emailAddress": "customer@example.com",
    "mobilePhone": "08011111111"
  },
  "address": {
    "addressNumber": "12", "streetName": "Broad Street",
    "town": "Lagos", "residentialCountry": "Nigeria",
    "residentialStateCode": "LA"
  }
}
```

> **DOCUMENTS REPLACE, THEY DO NOT APPEND**

> Including documents in a request replaces every existing document for that customer. To add a fourth file, resend all four. Files land in S3 at merchant-customers/{customerId}/documents/{folder}/{name}.

Auto-submit at CompletedSections = 63

All six bits set means 63, and that is the trigger. The service then:

- Loads every section and pulls the documents back from S3 as base64
- Builds the TieredData payload — Scheme = your MembershipId, SchemeCode = your MerchantId
- Calls CreateTieredCustomerAccountAsync
- On success: writes CrmAccountNumber, CustomerMembershipId, CrmApprovalStatus; status becomes Submitted; then fires wallet creation in the background using the customer's BVN, falling back to your corporate BVN if the customer has none
- On failure: status becomes Failed, FailureReason is stored, and 422 is returned with the CRM's reason

Listing your customers

`GET /api/MerchantOnboarding/merchant-customers/customers?page=1&pageSize=20`

```
{
  "data": [
    {
      "customerId": "...",
      "customerFirstName": "Alice",
      "customerSurname": "Smith",
      "customerEmail": "customer@example.com",
      "crmAccountNumber": "1234567890",
      "customerMembershipId": "MEM789012",
      "status": "Submitted",
      "createdAt": "2024-07-10T00:00:00Z"
    }
  ]
}
```

Customer approvals arrive on the same webhook as your own, routed by CustomerMembershipId, and are forwarded to your CallbackURL.

Part 5 — Products

With an approved account, money can move. All product endpoints are namespaced under /Product, use the base URL {{baseurl}}, and are authenticated with the same bearer token.

The response envelope

```
{
  "data": [ /* endpoint-specific payload */ ],
  "message": "Operation successful",
  "status": true,
  "statusCode": 200
}
```

Bulk transaction endpoints — every purchase and withdrawal — wrap their results with a transactionId alongside the data array:

```
{
  "transactionId": "eaa9487a-02af-4c87-8716-4e12e132d043",
  "data": [ /* per-request results */ ],
  "message": "",
  "status": true,
  "statusCode": 200
}
```

> **A 200 ON THE BATCH DOES NOT MEAN EVERY REQUEST SUCCEEDED**
>
> Each element of data carries its own status, purchaseId or withdrawalId, message, and an echo of the request. Inspect every element — do not stop at the envelope.

5.1 Virtual Accounts — B2B merchants only

> **READ THIS BEFORE YOU INTEGRATE**
>
> Virtual Accounts are strictly a B2B feature. Merchants operating under other models (B2B2C, B2C) cannot create or retrieve virtual accounts for their members — access is restricted at the authorisation layer. Do not build against these endpoints for a non-B2B flow.

Virtual accounts are dedicated bank account reservations tied to a member's investment identifier for a given product, used to receive and settle funds.

GET `/Product/VirtualAccounts`

POST `/Product/VirtualAccounts`

```
{ "productCode": "ARMMMF" }   // ARMMMF, TREASURYBILLS, BROKERAGE
```

```
{
  "data": [
    {
```




```
      "productCode": "ARMMMF",
      "investmentIdentifier": "1234567-ARMMMF-01",
      "bankName": "Sterling bank",
      "bankAccountName": "Jon Doe (ARMMMF)",
      "bankAccountNumber": "000000000"
    },
    { "bankName": "Wema bank", ... }
  ],
  "message": "Operation successful",
  "status": true,
  "statusCode": 200
}
```


You get several accounts across different partner banks — show them all and let the payer choose. All routes lead to the same investment. Keep the investmentIdentifier (format 1234567-ARMMMF-01): it is the same value as the accountNumber in the portfolio.

5.2 Mutual Funds

Browse the catalogue, read holdings, buy and redeem. Purchases and withdrawals are batch operations.

Method	Endpoint	Purpose
GET	`/Product/MutualFunds`	List mutual fund products
GET	`/Product/MutualFunds/Portfolio`	Get the member's holdings
POST	`/Product/mutualfunds/purchase/bulk`	Bulk mutual fund purchase
POST	`/Product/mutualfunds/withdraw/bulk`	Bulk mutual fund withdrawal

Browsing the catalogue

Field	Why it matters
productCode / identifier	What you pass everywhere else — e.g. ARMMMF, AGF, ARMFIF
canInvest / isActive	Check these first. If false, do not offer the fund.
mininumInitialInvestment	Minimum to open the investment. The misspelling is the API's, not a typo here.
mininumAdditionalInvestment	Minimum for subsequent purchases
productRate	Current yield or return rate
bidPrice / offerPrice	Buy and sell unit prices, where applicable — null for money-market funds priced at par
productRisk / productRiskDetails	Numeric risk score plus a human-readable profile (name, rating, description)
currency	Code, symbol, and USD conversion rate
factSheetLink	Link to the fund's fact sheet

Bulk purchase

POST `/Product/mutualfunds/purchase/bulk`


```
{
```

```
  "requests": [
    {
      "amount": 2000,
      "productCode": "ARMMMF",
      "membershipId": "3005575"
    }
  ]
}
```

```
{
  "transactionId": "eaa9487a-02af-4c87-8716-4e12e132d043",
  "data": [
    {
      "status": true,
      "purchaseId": "f7b38e5c-1670-4a4e-b7e1-daf43a6aa8a1",
      "message": "",
      "request": {
        "membershipId": "3005575",
        "reInvestDividends": true,
        "amount": 2000,
        "productCode": "ARMMMF"
      }
    }
  ],
  "message": "",
  "status": true,
  "statusCode": 200
}
```

Bulk withdraw

POST `/Product/mutualfunds/withdraw/bulk`

```
{
  "requests": [
    {
      "amount": 10000,
      "productCode": "ARMMMF",
      "membershipId": "3005575",
      "productAccountNumber": "3005575-AGF-01"
    }
  ]
}
```

The response mirrors the purchase shape, with a withdrawalId in place of purchaseId.

Reading the portfolio

GET `/Product/MutualFunds/Portfolio`

Field	Read it as
units / currentPrice	Units held and the current unit price
marketValue	Current market value of the holding
currentBalance	Validate withdrawals against this figure
bookBalance	Book value — it will differ from market value; do not show it as the headline number

Field	Read it as
purchaseUnits / pendingInflow	Money in, not yet settled
redemptionUnits / pendingOutflow	Withdrawals in flight
accruedInterest	Earned but not yet credited
accountNumber	The product account number — pass this as productAccountNumber when withdrawing
reInvest	Whether dividends are set to reinvest
reservedAccounts	Virtual accounts reserved for this holding (B2B merchants only)

DO NOT RECONSTRUCT ACCOUNT NUMBERS BY HAND

Read productAccountNumber from the portfolio. A number that does not belong to the authenticated membership is the most common cause of a rejected withdrawal.

5.3 Securities

A brokerage surface: discover instruments, fund the cash account, place batched orders, then manage them.

Discover

Method	Endpoint	Purpose
GET	`/Product/Securities/Stocks`	List tradable stocks
GET	`/Product/Securities/TreasuryBills`	List available treasury bills
GET	`/Product/Securities/StockRecommendation/:Query`	Search stock recommendations
GET	`/Product/Securities/MarketMovers`	Top gainers and losers for the day
GET	`/Product/Securities/TopTradedStocks`	Most actively traded stocks
GET	`/Product/Securities/MarketData/:SecId/:MarketCode`	Live market data for one security
GET	`/Product/Securities/ChartData`	Historical price data. SecId and MarketCode required; Range, StartDate, EndDate and DateRange optional

Fund and trade

Method	Endpoint	Purpose
POST	`/Product/Securities/CreditBrokerage/bulk`	Credit the brokerage cash account before trading. Needs amount, membershipId, cashAccountId

Method	Endpoint	Purpose
POST	/Product/Securities/Stocks/Purchase/Bulk	Bulk equity buy and sell orders
POST	/Product/Securities/DiscountedInstrument/Purchase/Bulk	Bulk treasury bill purchase. Needs faceValue, membershipId, and the instrument id

A stock order — MARKET and LIMIT in one batch

```
{
  "requests": [
    {
      "priceType": "MARKET",
      "orderType": "BUY",
      "orderTermName": "GOOD_FOR_A_DAY",
      "quantityRequested": 20,
      "secId": "NESTLE",
      "membershipId": "3001550"
    },
    {
      "priceType": "LIMIT",
      "orderType": "BUY",
      "LiMitPrice": 200,
      "orderTermName": "GOOD_FOR_A_DAY",
      "quantityRequested": 200,
      "secId": "GTCO",
      "membershipId": "3001550"
    }
  ]
}
```

THE LIMIT PRICE FIELD IS SPELT THREE WAYS IN THE SPEC

The field table calls it LimitPrice, the request example sends LiMitPrice, and the response echoes limitPrice. It is required for LIMIT orders and ignored for MARKET orders — confirm the accepted casing with the API team before you build against it.

Pull orderTermName values from GET /Product/Securities/TradeOrderTerms rather than hard-coding GOOD_FOR_A_DAY, and read enums such as order types, price types and asset types from GET /Product/Securities/Constants.

Manage

Method	Endpoint	Purpose
GET	/Product/Securities/Valuation?membershipId=…	Securities portfolio valuation for a member
GET	/Product/Securities/Orders?Page=1&Size=20	Paginated orders. Filter by Status, OrderType; sort with OrderBy
GET	/Product/Securities/TradeOrderTerms	Valid order term types, e.g. GOOD_FOR_A_DAY
GET	/Product/Securities/Constants	Securities lookup constants and enums

Method	Endpoint	Purpose
POST	`/Product/Securities/CancelTrade/:OrderId`	Cancel a previously placed order

Part 6 — Reference

Status values

Merchant onboarding	Meaning	Customer onboarding	Meaning
NotStarted	Created, nothing saved	NotStarted	Draft, nothing saved
InProgress	One or more sections saved	InProgress	At least one section saved
Submitted	Sent to CRM	Submitted	Accepted by CRM
Approved	Approval webhook received	Failed	CRM submission failed
Rejected	Rejection webhook received	—	—
UnderReview	Admin raised a document rejection	—	—

Merchant sections are bits 0–4 (Corporate, Contact, Signatories, Mandate, Documents). Customer sections are bits 0–5 — all six set equals 63, which triggers auto-submit.

Error codes you will actually see

Code	What happened
400	Bad payload, invalid credentials, invalid or expired OTP, or an account number that is not yours
401	OTP not validated for this email, or a missing / expired token
404	Webhook membershipId matched neither a customer nor a merchant
409	Business name or email already registered
422	CRM rejected the submission. The message carries the CRM's reason — read it, fix it, resubmit.

Endpoint index — registration, onboarding, webhooks

Stage	Endpoints
Register	POST /api/v1/merchant/validate · POST /api/v1/otp/validate · POST /api/v1/merchant/register
Log in	POST /api/v1/merchant/login
Onboard	POST /create · /corporate · /contact-bank · /signatories · /mandate-declarations · /documents · /onboard-merchant · /submit
Track and fix	GET /progress · /full-application · /documents/rejected · POST /documents/reject (admin)
Lookups	GET /average-turnover · /business-categories · /industry · /establishment-years · /number-of-employees · /accounts/lookup · POST /accounts/bulk · /accounts/bulk/local
Customers	POST /merchant-customers/onboard · GET /merchant-customers/customers
Webhooks	POST /merchant-approval · POST /merchant-crm-webhook

Endpoint index — products

#	Method	Endpoint	Merchant type
1	GET	/Product/VirtualAccounts	B2B only
2	POST	/Product/VirtualAccounts	B2B only
3	GET	/Product/Securities/Valuation	All
4	POST	/Product/Securities/CancelTrade/:OrderId	All
5	GET	/Product/Securities/TradeOrderTerms	All
6	GET	/Product/Securities/StockRecommendation/:Query	All
7	GET	/Product/Securities/Stocks	All
8	GET	/Product/Securities/TreasuryBills	All
9	GET	/Product/Securities/ChartData	All
10	GET	/Product/Securities/Orders	All
11	GET	/Product/Securities/Constants	All
12	GET	/Product/Securities/MarketMovers	All
13	GET	/Product/Securities/TopTradedStocks	All
14	GET	/Product/Securities/MarketData/:SecId/:MarketCode	All
15	POST	/Product/Securities/DiscountedInstrument/Purchase/Bulk	All
16	POST	/Product/Securities/Stocks/Purchase/Bulk	All
17	POST	/Product/Securities/CreditBrokerage/bulk	All
18	GET	/Product/MutualFunds	All
19	GET	/Product/MutualFunds/Portfolio	All
20	POST	/Product/mutualfunds/purchase/bulk	All
21	POST	/Product/mutualfunds/withdraw/bulk	All

Integration checklist

- Secret key captured at registration and stored securely
- callbackUrl registered and its handler live — it is how approvals reach you
- Token refreshed before expiresAt
- merchantEmail in every body matches the JWT email claim
- Dropdowns populated from the lookup endpoints, not hard-coded
- Document upload treated as async — confirmed via /progress, not the 200
- Customer documents array always sent complete (it replaces, not appends)
- 422 responses surfaced to the user with the CRM's actual reason
- Merchant type confirmed before building against Virtual Accounts

- canInvest and isActive checked before showing a fund
- Every element of a bulk response inspected, not just the batch status
- Withdrawals validated against currentBalance, using productAccountNumber from the portfolio
- Limit-price field casing confirmed with the API team